

 Objectif

Le but de ce TP est de poursuivre la découverte de la programmation via sockets, mais cette fois en utilisant le protocole TCP

Exercice 1 *Serveur TCP*

Nous avons vu la manipulation de socket UDP, avec l'envoi et la réception de message. Nous allons maintenant utiliser les sockets TCP, qui utilise un flux de données.

Le serveur suivant contient un compteur dont on peut obtenir la valeur en lui envoyant la requête get.

1.1 Recopier ce code, et consulter la documentation python pour comprendre les différentes instructions, en particulier la création de sockets en TCP, la méthode makefile, et les méthodes de lecture et d'écriture.

<https://docs.python.org/3/library/socket.html>

<https://docs.python.org/3/glossary.html#term-file-object>

```
1  #!/usr/bin/python3
2
3  import socket
4
5  class Server :
6
7      def __init__(self):
8          self.counter=0
9
10     def mainServer(self,port):
11         sock = socket.socket()
12         sock.bind(("0.0.0.0",port))
13         sock.setsockopt(socket.SOL_SOCKET,socket.SO_REUSEADDR,1)
14         sock.listen(10)
15         while True:
16             cli,addr = sock.accept()
17             sess = Session(self,cli)
18             sess.mainSession()
19
20 class Session:
21
22     def __init__(self,server,sock):
23         self.server = server
24         self.socket = sock
25         self.file = sock.makefile(mode="rw")
26
27     def mainSession(self):
28         while True:
29             line = self.file.readline().strip()
30             if line == "get":
31                 self.file.write("val %d\n" % self.server.counter )
32                 self.file.flush()
33             elif line == "quit":
34                 self.file.write("quit\n")
35                 self.file.flush()
36                 break
37             else:
38                 self.file.write("err\n")
```

```
39         self.file.flush()
40
41         self.file.close()
42         self.socket.shutdown(socket.SHUT_RDWR)
43         self.socket.close()
44
45
46 Server().mainServer(5555)
```

Tester votre serveur avec netcat
netcat localhost 5555

Exercice 2 *Client*

Voici un exemple de code client.

```
1  #!/usr/bin/python3
2  import socket
3
4  def client (host , port ):
5      sock = socket.socket()
6      sock.connect((host, port))
7      f = sock.makefile(mode="rw")
8      f.write("get\n")
9      f.flush()
10     print (f.readline(), end=" ")
11     f.write("quit\n")
12     f.flush()
13     print(f.readline(), end=" ")
14     f.close()
15     sock.shutdown(socket.SHUT_RDWR )
16     sock.close()
17
18 client ("localhost", 5555)
```

2.1 Recopiez le code et tester votre serveur.

2.2 Modifier le code du client pour qu'il lise sur l'entrée standard la commande du client, et l'envoi eu serveur, tant que l'utilisateur n'entre pas quit. Pensez à gérer les instructions non prévues par le serveur.

Exercice 3 *Extension du serveur*

3.1 Ajoutez les opérations suivantes sur le serveur

- incr incrémente le compteur et retourne la nouvelle valeur
- decr décrémente le compteur et retourne la nouvelle valeur
- add N ajoute N au compteur et retourne la nouvelle valeur